

**12576 - Nisanth Saravanan**  
**L1 ASP .NET &MVC Assessmet**

**Smart Clinic Appointment System**

This web application streamlines appointment scheduling for multi-specialty clinics. It enables patients to book appointments based on doctor availability, while doctors and administrators manage schedules and patient records. Built using ASP.NET Core MVC, the system integrates role-based access, calendar management, and automated notifications via background jobs. It enhances operational efficiency and improves patient experience through a responsive and secure interface. Entities:

Patient: PatientId, Name, Email, Phone, DOB, Gender

Doctor: DoctorId, Name, Specialization, Availability

Appointment: AppointmentId, PatientId, DoctorId, Date, Time, Status

Notification: NotificationId, UserId, Message, SentDate

**Task:**

Perform W3H analysis on the given Case Study.

Use the Case Study to implement ASP.Net MVC Core Web application Code-First Approach.

Connect to MySql server and create database and tables as needed and implement table relationships [one to one or one to many or many to many] as required.

Finally, perform all CRUD operations using ASP.MVC Core in the Entity Framework on the given entity. Make the user interactive application with the support of Razor Pages.

- Add data
- Search data by id
- Update data by id
- Delete data by id
- Display all data

## W3H Analysis

| Project Title: Smart Clinic Appointment System                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What?                                                                                                                                                                                                                                                  | How?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <p><b>1. What are the various entities needed for this database?</b></p> <p><b>A: Required Entities:</b></p> <ul style="list-style-type: none"><li>a) Patient</li><li>b) Doctor</li><li>c) Appointment</li></ul>                                       | <p><b>1. Patient:</b></p> <p><b>a) Store Patient Data:</b></p> <p><b>Method 1: Use SQL DB.</b></p> <p>Method 2: Use List of Class.</p> <p>Method 3: Use Local file.</p> <p><b>b) Edit Patient Data:</b></p> <p><b>Method 1: Edit by ID.</b></p> <p>Method 2: Edit by Name.</p> <p>Method 3: Edit by range of From to To Date of Creation.</p> <p><b>c) Delete Patient Data:</b></p> <p><b>Method 1: Delete by ID.</b></p> <p>Method 2: Delete by Name.</p> <p>Method 3: Delete by range of From to To Date of Creation.</p> |
| <p><b>2. What are the necessary functionalities of Patient?</b></p> <p><b>A: Patient:</b></p> <ul style="list-style-type: none"><li>a) Store Patient Data.</li><li>b) Edit Patient Data.</li><li>c) Delete Patient Data.</li></ul>                     | <p><b>2. Doctor:</b></p> <p><b>a) Store Doctor Data:</b></p> <p><b>Method 1: Use SQL DB.</b></p> <p>Method 2: Use List of Class.</p> <p>Method 3: Use Local file.</p> <p><b>b) Edit Doctor Data:</b></p> <p><b>Method 1: Edit by ID.</b></p> <p>Method 2: Edit by Name.</p> <p>Method 3: Edit by range of From to To Date of Creation.</p> <p><b>c) Delete Doctor Data:</b></p>                                                                                                                                             |
| <p><b>3. What are the necessary functionalities of Doctor?</b></p> <p><b>A: Doctor:</b></p> <ul style="list-style-type: none"><li>a) Store Doctor Data.</li><li>b) Edit Doctor Data.</li><li>c) Delete Doctor Data.</li></ul>                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <p><b>4. What are the necessary functionalities of Appointment?</b></p> <p><b>A: Appointment:</b></p> <ul style="list-style-type: none"><li>a) Store Appointment Data.</li><li>b) Edit Appointment Data.</li><li>c) Delete Appointment Data.</li></ul> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <p><b>Method 1: Delete by ID.</b><br/> Method 2: Delete by Name.<br/> Method 3: Delete by range of From to To Date of Creation.</p> <p><b>3. Appointment:</b><br/> <b>a) Store Appointment Data:</b><br/> <b>Method 1: Use SQL DB.</b><br/> Method 2: Use List of Class.<br/> Method 3: Use Local file.<br/> <b>b) Edit Appointment Data:</b><br/> <b>Method 1: Edit by ID.</b><br/> Method 2: Edit by Name.<br/> Method 3: Edit by range of From to To Date of Creation.<br/> <b>c) Delete Appointment Data:</b><br/> <b>Method 1: Delete by ID.</b><br/> Method 2: Delete by Name.<br/> Method 3: Delete by range of From to To Date of Creation.</p> |
| <p><b>1. Patient:</b><br/> <b>a) Store Patient Data:</b><br/> <b>Method 1: Use SQL DB.</b><br/> A: Fits the use case and proves to be a highly effective method to achieve the desired outcome.<br/> <b>b) Edit Patient Data:</b><br/> <b>Method 1: Edit by ID.</b><br/> A: Fits the use case and proves to be a highly effective method to achieve the desired outcome.<br/> <b>c) Delete Patient Data:</b><br/> <b>Method 1: Delete by ID.</b><br/> A: Fits the use case and proves to be a highly effective method to achieve the</p> | <p><b>1. Patient:</b><br/> <b>a) Store Patient Data:</b><br/> Method 2: Use List of Class.<br/> Method 3: Use Local file.<br/> A: The methods are feasible, but they lack the effectiveness to be selected as the primary option.<br/> <b>b) Edit Patient Data:</b><br/> Method 2: Edit by Name.<br/> Method 3: Edit by range of From to To Date of Creation.<br/> A: The methods are feasible, but they lack the effectiveness to be selected as the primary option.<br/> <b>c) Delete Patient Data:</b></p>                                                                                                                                           |

desired outcome.

## **2. Doctor:**

### **a) Store Doctor Data:**

#### **Method 1: Use SQL DB.**

A: Fits the use case and proves to be a highly effective method to achieve the desired outcome.

### **b) Edit Doctor Data:**

#### **Method 1: Edit by ID.**

A: Fits the use case and proves to be a highly effective method to achieve the desired outcome.

### **c) Delete Doctor Data:**

#### **Method 1: Delete by ID.**

A: Fits the use case and proves to be a highly effective method to achieve the desired outcome.

## **3. Appointment:**

### **a) Store Appointment Data:**

#### **Method 1: Use SQL DB.**

A: Fits the use case and proves to be a highly effective method to achieve the desired outcome.

### **b) Edit Appointment Data:**

#### **Method 1: Edit by ID.**

A: Fits the use case and proves to be a highly effective method to achieve the desired outcome.

### **c) Delete Appointment Data:**

#### **Method 1: Delete by ID.**

A: Fits the use case and proves to be a highly effective method to achieve the desired outcome.

## **Method 2: Delete by Name.**

## **Method 3: Delete by range of From to To Date of Creation.**

A: The methods are feasible, but they lack the effectiveness to be selected as the primary option.

## **2. Doctor:**

### **a) Store Doctor Data:**

#### **Method 2: Use List of Class.**

#### **Method 3: Use Local file.**

A: The methods are feasible, but they lack the effectiveness to be selected as the primary option.

### **b) Edit Doctor Data:**

#### **Method 2: Edit by Name.**

## **Method 3: Edit by range of From to To Date of Creation.**

A: The methods are feasible, but they lack the effectiveness to be selected as the primary option.

### **c) Delete Doctor Data:**

#### **Method 2: Delete by Name.**

## **Method 3: Delete by range of From to To Date of Creation.**

A: The methods are feasible, but they lack the effectiveness to be selected as the primary option.

## **3. Appointment:**

### **a) Store Appointment Data:**

#### **Method 2: Use List of Class.**

#### **Method 3: Use Local file.**

A: The methods are feasible, but they lack the effectiveness to be selected as the primary option.

|             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             | <p><b>b) Edit Appointment Data:</b></p> <p>Method 2: Edit by Name.</p> <p>Method 3: Edit by range of From to To Date of Creation.</p> <p>A: The methods are feasible, but they lack the effectiveness to be selected as the primary option.</p> <p><b>c) Delete Appointment Data:</b></p> <p>Method 2: Delete by Name.</p> <p>Method 3: Delete by range of From to To Date of Creation.</p> <p>A: The methods are feasible, but they lack the effectiveness to be selected as the primary option.</p> |
| <b>Why?</b> | <b>Why Not?</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

# Code Models

## Patient

```
using System;
using System.Collections.Generic;

namespace AppointmentManager.Models;

public partial class Patient
{
    public int PatientId { get; set; }

    public string? PatientName { get; set; }

    public string? PatientEmail { get; set; }

    public int? PatientPhone { get; set; }

    public DateOnly? PatientDob { get; set; }

    public string? PatientGender { get; set; }

    public virtual Appointment? Appointment { get; set; }
}
```

# Doctor

```
using System;
using System.Collections.Generic;

namespace AppointmentManager.Models;

public partial class Doctor
{
    public int DoctorId { get; set; }

    public string? DoctorName { get; set; }

    public string? Specialization { get; set; }

    public string? Availability { get; set; }

    public virtual Appointment? Appointment { get; set; }
}
```

# Appointment

```
using System;
using System.Collections.Generic;

namespace AppointmentManager.Models;

public partial class Appointment
{
    public int AppointmentId { get; set; }

    public int? PatientId { get; set; }

    public int? DoctorId { get; set; }

    public DateOnly? DateofAppoint { get; set; }

    public TimeOnly? TimeofAppoint { get; set; }

    public string? StatusofAppoint { get; set; }

    public virtual Doctor? Doctor { get; set; }

    public virtual Patient? Patient { get; set; }
}
```



# Controllers

## Patient

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using AppointmentManager.Data;
using AppointmentManager.Models;

namespace AppointmentManager.Controllers
{
    public class PatientController : Controller
    {
        private readonly ApplicationDbContext _context;

        public PatientController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: Patient
        public async Task<IActionResult> Index()
        {
            return View(await _context.Patients.ToListAsync());
        }

        // GET: Patient/Details/5
        public async Task<IActionResult> Details(int? id)
        {
            if (id == null)
            {
                return NotFound();
            }

            var patient = await _context.Patients
                .FirstOrDefaultAsync(m => m.PatientId == id);
            if (patient == null)
            {
                return NotFound();
            }

            return View(patient);
        }

        // GET: Patient/Create
        public IActionResult Create()
        {
            return View();
        }

        // POST: Patient/Create
        // To protect from overposting attacks, enable the specific properties you
        want to bind to.
        // For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
        [HttpPost]
        [ValidateAntiForgeryToken]
```

```

        public async Task<IActionResult>
Create([Bind("PatientId,PatientName,PatientEmail,PatientPhone,PatientDob,PatientGender
")] Patient patient)
    {
        if (ModelState.IsValid)
        {
            if (patient.PatientGender == "" || patient.PatientGender == null) {
                return RedirectToAction(nameof(Create));
            }

            _context.Add(patient);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }
        return View(patient);
    }

// GET: Patient/Edit/5
public async Task<IActionResult> Edit(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var patient = await _context.Patients.FindAsync(id);
    if (patient == null)
    {
        return NotFound();
    }
    return View(patient);
}

// POST: Patient/Edit/5
// To protect from overposting attacks, enable the specific properties you
want to bind to.
// For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id,
[Bind("PatientId,PatientName,PatientEmail,PatientPhone,PatientDob,PatientGender")]
Patient patient)
{
    if (id != patient.PatientId)
    {
        return NotFound();
    }

    if (ModelState.IsValid)
    {
        try
        {
            if (patient.PatientGender == "" || patient.PatientGender == null)
            {
                return RedirectToAction(nameof(Edit));
            }

            _context.Update(patient);
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!PatientExists(patient.PatientId))

```

```

        {
            return NotFound();
        }
        else
        {
            throw;
        }
    }
    return RedirectToAction(nameof(Index));
}
return View(patient);
}

// GET: Patient/Delete/5
public async Task<IActionResult> Delete(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var patient = await _context.Patients
        .FirstOrDefaultAsync(m => m.PatientId == id);
    if (patient == null)
    {
        return NotFound();
    }

    return View(patient);
}

// POST: Patient/Delete/5
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var patient = await _context.Patients.FindAsync(id);
    if (patient != null)
    {
        _context.Patients.Remove(patient);
    }

    await _context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}

private bool PatientExists(int id)
{
    return _context.Patients.Any(e => e.PatientId == id);
}
}

```

# Doctor

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using AppointmentManager.Data;
using AppointmentManager.Models;

namespace AppointmentManager.Controllers
{
    public class DoctorController : Controller
    {
        private readonly ApplicationDbContext _context;

        public DoctorController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: Doctor
        public async Task<IActionResult> Index()
        {
            return View(await _context.Doctors.ToListAsync());
        }

        // GET: Doctor/Details/5
        public async Task<IActionResult> Details(int? id)
        {
            if (id == null)
            {
                return NotFound();
            }

            var doctor = await _context.Doctors
                .FirstOrDefaultAsync(m => m.DoctorId == id);
            if (doctor == null)
            {
                return NotFound();
            }

            return View(doctor);
        }

        // GET: Doctor/Create
        public IActionResult Create()
        {
            return View();
        }

        // POST: Doctor/Create
        // To protect from overposting attacks, enable the specific properties you
        want to bind to.
        // For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult>
        Create([Bind("DoctorId,DoctorName,Specialization,Availability")] Doctor doctor)
        {
            if (ModelState.IsValid)
            {

```

```

        if(doctor.Availability == "" || doctor.Availability == null)
        {
            return RedirectToAction(nameof(Create));
        }

        _context.Add(doctor);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    return View(doctor);
}

// GET: Doctor/Edit/5
public async Task<IActionResult> Edit(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var doctor = await _context.Doctors.FindAsync(id);
    if (doctor == null)
    {
        return NotFound();
    }
    return View(doctor);
}

// POST: Doctor/Edit/5
// To protect from overposting attacks, enable the specific properties you
want to bind to.
// For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id,
[Bind("DoctorId,DoctorName,Specialization,Availability")] Doctor doctor)
{
    if (id != doctor.DoctorId)
    {
        return NotFound();
    }

    if (ModelState.IsValid)
    {
        try
        {
            if (doctor.Availability == "" || doctor.Availability == null)
            {
                return RedirectToAction(nameof(Edit));
            }

            _context.Update(doctor);
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!DoctorExists(doctor.DoctorId))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }
    }
}

```

```

        }
    }
    return RedirectToAction(nameof(Index));
}
return View(doctor);
}

// GET: Doctor/Delete/5
public async Task<IActionResult> Delete(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var doctor = await _context.Doctors
        .FirstOrDefaultAsync(m => m.DoctorId == id);
    if (doctor == null)
    {
        return NotFound();
    }

    return View(doctor);
}

// POST: Doctor/Delete/5
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var doctor = await _context.Doctors.FindAsync(id);
    if (doctor != null)
    {
        _context.Doctors.Remove(doctor);
    }

    await _context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}

private bool DoctorExists(int id)
{
    return _context.Doctors.Any(e => e.DoctorId == id);
}
}
}

```

# Appointment

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using AppointmentManager.Data;
using AppointmentManager.Models;

namespace AppointmentManager.Controllers
{
    public class AppointmentController : Controller
    {
        private readonly AppDbContext _context;

        public AppointmentController(AppDbContext context)
        {
            _context = context;
        }

        // GET: Appointment
        public async Task<IActionResult> Index()
        {
            var appDbContext = _context.Appointments.Include(a => a.Doctor).Include(a
=> a.Patient);
            return View(await appDbContext.ToListAsync());
        }

        // GET: Appointment/Details/5
        public async Task<IActionResult> Details(int? id)
        {
            if (id == null)
            {
                return NotFound();
            }

            var appointment = await _context.Appointments
                .Include(a => a.Doctor)
                .Include(a => a.Patient)
                .FirstOrDefaultAsync(m => m.AppointmentId == id);
            if (appointment == null)
            {
                return NotFound();
            }

            return View(appointment);
        }

        // GET: Appointment/Create
        public IActionResult Create()
        {
            ViewData["DoctorId"] = new SelectList(_context.Doctors, "DoctorId",
"DoctorId");
            ViewData["PatientId"] = new SelectList(_context.Patients, "PatientId",
"PatientId");
            ViewData["DoctorName"] = new SelectList(_context.Doctors, "DoctorId",
"DoctorName");
            ViewData["PatientName"] = new SelectList(_context.Patients, "PatientId",
"PatientName");
            return View();
        }
    }
}
```

```

        // POST: Appointment/Create
        // To protect from overposting attacks, enable the specific properties you
        want to bind to.
        // For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult>
Create([Bind("AppointmentId, PatientId, DoctorId, DateofAppoint, TimeofAppoint, StatusofApp
oint")] Appointment appointment)
    {
        if (ModelState.IsValid)
        {
            if(appointment.StatusofAppoint == "" || appointment.StatusofAppoint ==
null)
            {
                return RedirectToAction(nameof(Create));
            }

            _context.Add(appointment);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }
        ViewData["DoctorId"] = new SelectList(_context.Doctors, "DoctorId",
"DoctorId", appointment.DoctorId);
        ViewData["PatientId"] = new SelectList(_context.Patients, "PatientId",
"PatientId", appointment.PatientId);
        return View(appointment);
    }

    // GET: Appointment/Edit/5
    public async Task<IActionResult> Edit(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var appointment = await _context.Appointments.FindAsync(id);
        if (appointment == null)
        {
            return NotFound();
        }
        ViewData["DoctorName"] = new SelectList(_context.Doctors, "DoctorId",
"DoctorName");
        ViewData["PatientName"] = new SelectList(_context.Patients, "PatientId",
"PatientName");
        ViewData["DoctorId"] = new SelectList(_context.Doctors, "DoctorId",
"DoctorId", appointment.DoctorId);
        ViewData["PatientId"] = new SelectList(_context.Patients, "PatientId",
"PatientId", appointment.PatientId);
        return View(appointment);
    }

    // POST: Appointment/Edit/5
    // To protect from overposting attacks, enable the specific properties you
    want to bind to.
    // For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(int id,
[Bind("AppointmentId, PatientId, DoctorId, DateofAppoint, TimeofAppoint, StatusofAppoint")]
Appointment appointment)
    {

```



```

        if (id != appointment.AppointmentId)
        {
            return NotFound();
        }

        if (ModelState.IsValid)
        {
            try
            {
                if (appointment.StatusofAppoint == "" ||
appointment.StatusofAppoint == null)
                {
                    return RedirectToAction(nameof(Edit));
                }

                _context.Update(appointment);
                await _context.SaveChangesAsync();
            }
            catch (DbUpdateConcurrencyException)
            {
                if (!AppointmentExists(appointment.AppointmentId))
                {
                    return NotFound();
                }
                else
                {
                    throw;
                }
            }
            return RedirectToAction(nameof(Index));
        }
        ViewData["DoctorId"] = new SelectList(_context.Doctors, "DoctorId",
"DoctorId", appointment.DoctorId);
        ViewData["PatientId"] = new SelectList(_context.Patients, "PatientId",
"PatientId", appointment.PatientId);
        return View(appointment);
    }

    // GET: Appointment/Delete/5
    public async Task<IActionResult> Delete(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var appointment = await _context.Appointments
            .Include(a => a.Doctor)
            .Include(a => a.Patient)
            .FirstOrDefaultAsync(m => m.AppointmentId == id);
        if (appointment == null)
        {
            return NotFound();
        }

        return View(appointment);
    }

    // POST: Appointment/Delete/5
    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int id)
    {
        var appointment = await _context.Appointments.FindAsync(id);

```

```
        if (appointment != null)
        {
            _context.Appointments.Remove(appointment);
        }

        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }

    private bool AppointmentExists(int id)
    {
        return _context.Appointments.Any(e => e.AppointmentId == id);
    }
}
```

# DbContext

```
using System;
using System.Collections.Generic;
using AppointmentManager.Models;
using Microsoft.EntityFrameworkCore;

namespace AppointmentManager.Data;

public partial class AppDbContext : DbContext
{
    public AppDbContext()
    {
    }

    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public virtual DbSet<Appointment> Appointments { get; set; }

    public virtual DbSet<Doctor> Doctors { get; set; }

    public virtual DbSet<Patient> Patients { get; set; }
}
```

# Views

## Home/Index

```
@{
    ViewData["Title"] = "Home Page";
}

<div class="text-center">
    <h1 class="display-4">Welcome To Rz Hozpitalz!</h1>
    <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
    incididunt ut labore et dolore magna aliqua.
        Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut
    aliquip ex ea commodo consequat.
        Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore
    eu fugiat nulla pariatur.
        Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia
    deserunt mollit anim id est laborum.</p>
    <br />
    <br />
    <button class="btn btn-dark"><a class="text-decoration-none text-light" asp-
    area="" asp-controller="Doctor" asp-action="Index">View Doctors</a></button>
    <button class="btn btn-dark"><a class="text-decoration-none text-light" asp-
    area="" asp-controller="Appointment" asp-action="Index">View Appointments</a></button>
    <button class="btn btn-dark"><a class="text-decoration-none text-light" asp-
    area="" asp-controller="Patient" asp-action="Index">View Patients</a></button>
</div>
```

# Patient/Index

```
@model IEnumerable<AppointmentManager.Models.Patient>

@{
    ViewData["Title"] = "Index";
}

<h1>Index</h1>

<p>
    <button class="btn btn-dark"><a class="text-decoration-none text-light" asp-
action="Create">Create New Record</a></button>
</p>
<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.PatientName)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.PatientEmail)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.PatientPhone)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.PatientDob)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.PatientGender)
            </th>
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model) {
            <tr>
                <td>
                    @Html.DisplayFor(modelItem => item.PatientName)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.PatientEmail)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.PatientPhone)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.PatientDob)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.PatientGender)
                </td>
                <td>
                    <button class="btn btn-primary"><a class="text-decoration-none text-
light" asp-action="Edit" asp-route-id="@item.PatientId">Edit</a></button> |
                    <button class="btn btn-info"><a class="text-decoration-none text-
light" asp-action="Details" asp-route-id="@item.PatientId">Details</a></button> |
                    <button class="btn btn-danger"><a class="text-decoration-none text-
light" asp-action="Delete" asp-route-id="@item.PatientId">Delete</a></button>
                </td>
            </tr>
        }
    </tbody>
</table>
```

```
</table>
```

## Patient/Create

```
@model AppointmentManager.Models.Patient
```

```
@{  
    ViewData["Title"] = "Create";  
}
```

```
<h1>Create</h1>
```

```
<h4>Patient</h4>
```

```
<hr />
```

```
<div class="row">
```

```
    <div class="col-md-4">
```

```
        <form asp-action="Create">
```

```
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
```

```
            <div class="form-group">
```

```
                <label asp-for="PatientName" class="control-label">Patient
```

```
Name:</label>
```

```
                <input asp-for="PatientName" class="form-control" />
```

```
                <span asp-validation-for="PatientName" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="PatientEmail" class="control-label">Email:</label>
```

```
                <input asp-for="PatientEmail" class="form-control" />
```

```
                <span asp-validation-for="PatientEmail" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="PatientPhone" class="control-label">Contact
```

```
Number:</label>
```

```
                <input asp-for="PatientPhone" class="form-control" />
```

```
                <span asp-validation-for="PatientPhone" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="PatientDob" class="control-label">Date of Birth:
```

```
(YYYY-MM-DD Format)</label>
```

```
                <input asp-for="PatientDob" class="form-control" />
```

```
                <span asp-validation-for="PatientDob" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="PatientGender" class="control-label">Gender:</label>
```

```
                <select asp-for="PatientGender" class="form-control">
```

```
                    <option value="">-- Select Gender --</option>
```

```
                    <option value="Male">Male</option>
```

```
                    <option value="Female">Female</option>
```

```
                    <option value="Other">Other</option>
```

```
                </select>
```

```
                <span asp-validation-for="PatientGender" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <input type="submit" value="Create" class="btn btn-primary" />
```

```
            </div>
```

```
        </form>
```

```
    </div>
```

```
</div>
```

```
<div>
```

```
    <a asp-action="Index">Back to List</a>
```

```
</div>
```

```
@section Scripts {
```

```
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
```

```
}
```

# Patient/Edit

```
@model AppointmentManager.Models.Patient

@{
    ViewData["Title"] = "Edit";
}

<h1>Edit</h1>

<h4>Patient</h4>
<hr />
<div class="row">
    <div class="col-md-4">
        <form asp-action="Edit">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <input type="hidden" asp-for="PatientId" />
            <div class="form-group">
                <label asp-for="PatientName" class="control-label">Patient
Name:</label>
                <input asp-for="PatientName" class="form-control" />
                <span asp-validation-for="PatientName" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="PatientEmail" class="control-label">Email:</label>
                <input asp-for="PatientEmail" class="form-control" />
                <span asp-validation-for="PatientEmail" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="PatientPhone" class="control-label">Contact
Number:</label>
                <input asp-for="PatientPhone" class="form-control" />
                <span asp-validation-for="PatientPhone" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="PatientDob" class="control-label">Date of Birth:
(YYYY-MM-DD Format)</label>
                <input asp-for="PatientDob" class="form-control" />
                <span asp-validation-for="PatientDob" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="PatientGender" class="control-label">Gender:</label>
                <select asp-for="PatientGender" class="form-control">
                    <option value="">-- Select Gender --</option>
                    <option value="Male">Male</option>
                    <option value="Female">Female</option>
                    <option value="Other">Other</option>
                </select>
                <span asp-validation-for="PatientGender" class="text-danger"></span>
            </div>
            <div class="form-group">
                <input type="submit" value="Save" class="btn btn-primary" />
            </div>
        </form>
    </div>
</div>

<div>
    <a asp-action="Index">Back to List</a>
</div>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
```

```
}
```

## Patient/Delete

```
@model AppointmentManager.Models.Patient

@{
    ViewData["Title"] = "Delete";
}

<h1>Delete</h1>

<h3>Are you sure you want to delete this?</h3>
<div>
    <h4>Patient</h4>
    <hr />
    <dl class="row">
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.PatientName)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.PatientName)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.PatientEmail)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.PatientEmail)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.PatientPhone)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.PatientPhone)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.PatientDob)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.PatientDob)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.PatientGender)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.PatientGender)
        </dd>
    </dl>

    <form asp-action="Delete">
        <input type="hidden" asp-for="PatientId" />
        <input type="submit" value="Delete" class="btn btn-danger" /> |
        <a asp-action="Index">Back to List</a>
    </form>
</div>
```



# Patient/Details

```
@model AppointmentManager.Models.Patient

@{
    ViewData["Title"] = "Details";
}

<h1>Details</h1>

<div>
    <h4>Patient</h4>
    <hr />
    <dl class="row">
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.PatientName)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.PatientName)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.PatientEmail)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.PatientEmail)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.PatientPhone)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.PatientPhone)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.PatientDob)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.PatientDob)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.PatientGender)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.PatientGender)
        </dd>
    </dl>
</div>
<div>
    <a asp-action="Edit" asp-route-id="@Model?.PatientId">Edit</a> |
    <a asp-action="Index">Back to List</a>
</div>
```

# Doctor/Index

```
@model IEnumerable<AppointmentManager.Models.Doctor>

@{
    ViewData["Title"] = "Index";
}

<h1>Index</h1>

<p>
    <button class="btn btn-dark"><a class="text-decoration-none text-light" asp-
action="Create">Create New Record</a></button>
</p>
<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.DoctorName)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Specialization)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Availability)
            </th>
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model) {
            <tr>
                <td>
                    @Html.DisplayFor(modelItem => item.DoctorName)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.Specialization)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.Availability)
                </td>
                <td>
                    <button class="btn btn-primary"><a class="text-decoration-none
text-light" asp-action="Edit" asp-route-id="@item.DoctorId">Edit</a></button> |
                    <button class="btn btn-info"><a class="text-decoration-none text-
light" asp-action="Details" asp-route-id="@item.DoctorId">Details</a></button> |
                    <button class="btn btn-danger"><a class="text-decoration-none
text-light" asp-action="Delete" asp-route-id="@item.DoctorId">Delete</a></button>
                </td>
            </tr>
        }
    </tbody>
</table>
```

# Doctor/Create

```
@model AppointmentManager.Models.Doctor
```

```
@{  
    ViewData["Title"] = "Create";  
}
```

```
<h1>Create</h1>
```

```
<h4>Doctor</h4>
```

```
<hr />
```

```
<div class="row">
```

```
    <div class="col-md-4">
```

```
        <form asp-action="Create">
```

```
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
```

```
            <div class="form-group">
```

```
                <label asp-for="DoctorName" class="control-label">Doctor Name:</label>
```

```
                <input asp-for="DoctorName" class="form-control" />
```

```
                <span asp-validation-for="DoctorName" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="Specialization" class="control-label">Specialization:</label>
```

```
                <input asp-for="Specialization" class="form-control" />
```

```
                <span asp-validation-for="Specialization" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="Availability" class="control-label">Availability:</label>
```

```
                <select asp-for="Availability" class="form-control">
```

```
                    <option value="">-- Select Availability --</option>
```

```
                    <option value="Available">Available</option>
```

```
                    <option value="Unavailable">Unavailable</option>
```

```
                </select>
```

```
                <span asp-validation-for="Availability" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <input type="submit" value="Create" class="btn btn-primary" />
```

```
            </div>
```

```
        </form>
```

```
    </div>
```

```
</div>
```

```
<div>
```

```
    <a asp-action="Index">Back to List</a>
```

```
</div>
```

```
@section Scripts {
```

```
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
```

```
}
```

# Doctor/Edit

```
@model AppointmentManager.Models.Doctor
```

```
@{  
    ViewData["Title"] = "Edit";  
}
```

```
<h1>Edit</h1>
```

```
<h4>Doctor</h4>
```

```
<hr />
```

```
<div class="row">
```

```
    <div class="col-md-4">
```

```
        <form asp-action="Edit">
```

```
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
```

```
            <input type="hidden" asp-for="DoctorId" />
```

```
            <div class="form-group">
```

```
                <label asp-for="DoctorName" class="control-label">Doctor Name:</label>
```

```
                <input asp-for="DoctorName" class="form-control" />
```

```
                <span asp-validation-for="DoctorName" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="Specialization" class="control-label">Specialization:</label>
```

```
                <input asp-for="Specialization" class="form-control" />
```

```
                <span asp-validation-for="Specialization" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="Availability" class="control-label">Availability:</label>
```

```
                <select asp-for="Availability" class="form-control">
```

```
                    <option value="">-- Select Availability --</option>
```

```
                    <option value="Available">Available</option>
```

```
                    <option value="Unavailable">Unavailable</option>
```

```
                </select>
```

```
                <span asp-validation-for="Availability" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <input type="submit" value="Save" class="btn btn-primary" />
```

```
            </div>
```

```
        </form>
```

```
    </div>
```

```
</div>
```

```
<div>
```

```
    <a asp-action="Index">Back to List</a>
```

```
</div>
```

```
@section Scripts {
```

```
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
```

```
}
```

## Doctor/Delete

```
@model AppointmentManager.Models.Doctor

@{
    ViewData["Title"] = "Delete";
}

<h1>Delete</h1>

<h3>Are you sure you want to delete this?</h3>
<div>
    <h4>Doctor</h4>
    <hr />
    <dl class="row">
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.DoctorName)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.DoctorName)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Specialization)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Specialization)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Availability)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Availability)
        </dd>
    </dl>

    <form asp-action="Delete">
        <input type="hidden" asp-for="DoctorId" />
        <input type="submit" value="Delete" class="btn btn-danger" /> |
        <a asp-action="Index">Back to List</a>
    </form>
</div>
```

# Doctor/Details

```
@model AppointmentManager.Models.Doctor

@{
    ViewData["Title"] = "Details";
}

<h1>Details</h1>

<div>
    <h4>Doctor</h4>
    <hr />
    <dl class="row">
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.DoctorName)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.DoctorName)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Specialization)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Specialization)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Availability)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Availability)
        </dd>
    </dl>
</div>
<div>
    <a asp-action="Edit" asp-route-id="@Model?.DoctorId">Edit</a> |
    <a asp-action="Index">Back to List</a>
</div>
```

# Appointment/Index

```
@model IEnumerable<AppointmentManager.Models.Appointment>

@{
    ViewData["Title"] = "Index";
}

<h1>Index</h1>

<p>
    <button class="btn btn-dark"><a class="text-decoration-none text-light" asp-
action="Create">Create New Appointment</a></button>
</p>
<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.DateofAppoint)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.TimeofAppoint)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.StatusofAppoint)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Doctor)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Patient)
            </th>
            <th></th>
        </tr>
    </thead>
    <tbody>
@foreach (var item in Model) {
        <tr>
            <td>
                @Html.DisplayFor(modelItem => item.DateofAppoint)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.TimeofAppoint)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.StatusofAppoint)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Doctor.DoctorName)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Patient.PatientName)
            </td>
            <td>
                <button class="btn btn-primary"><a class="text-decoration-none
text-light" asp-action="Edit" asp-route-id="@item.AppointmentId">Edit</a></button> |
                <button class="btn btn-info"><a class="text-decoration-none text-
light" asp-action="Details" asp-route-id="@item.AppointmentId">Details</a></button> |
                <button class="btn btn-danger"><a class="text-decoration-none
text-light" asp-action="Delete" asp-route-id="@item.AppointmentId">Delete</a></button>
            </td>
        </tr>
    }
}
```

```
</tbody>
</table>
```

## Appointment/Create

```
@model AppointmentManager.Models.Appointment
```

```
@{
    ViewData["Title"] = "Create";
}
```

```
<h1>Create</h1>
```

```
<h4>Appointment</h4>
```

```
<hr />
```

```
<div class="row">
```

```
    <div class="col-md-4">
```

```
        <form asp-action="Create">
```

```
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
```

```
            <div class="form-group">
```

```
                <label asp-for="PatientId" class="control-label">Patient Name:</label>
```

```
                <select asp-for="PatientId" class="form-control" asp-
```

```
items="ViewBag.PatientName"></select>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="DoctorId" class="control-label">Doctor Name:</label>
```

```
                <select asp-for="DoctorId" class="form-control" asp-
```

```
items="ViewBag.DoctorName"></select>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="DateofAppoint" class="control-label">Date of
Appointment: (YYYY-MM-DD Format)</label>
```

```
                <input asp-for="DateofAppoint" class="form-control" />
```

```
                <span asp-validation-for="DateofAppoint" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="TimeofAppoint" class="control-label">Time of
Appointment: (24 Hour Format)</label>
```

```
                <input asp-for="TimeofAppoint" class="form-control" />
```

```
                <span asp-validation-for="TimeofAppoint" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="StatusofAppoint" class="control-label">Status of
Appointment:</label>
```

```
                <select asp-for="StatusofAppoint" class="form-control">
```

```
                    <option value="">-- Select Status --</option>
```

```
                    <option value="Pending">Pending</option>
```

```
                    <option value="Ongoing">Ongoing</option>
```

```
                    <option value="Cancelled">Cancelled</option>
```

```
                    <option value="Completed">Completed</option>
```

```
                </select>
```

```
                <span asp-validation-for="StatusofAppoint" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <input type="submit" value="Create" class="btn btn-primary" />
```

```
            </div>
```

```
        </form>
```

```
    </div>
```

```
</div>
```

```
<div>
```

```
    <a asp-action="Index">Back to List</a>
```

```
</div>
```



```
@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}
```

## Appointment/Edit

```
@model AppointmentManager.Models.Appointment
```

```
@{
    ViewData["Title"] = "Edit";
}
```

```
<h1>Edit</h1>
```

```
<h4>Appointment</h4>
```

```
<hr />
```

```
<div class="row">
```

```
    <div class="col-md-4">
```

```
        <form asp-action="Edit">
```

```
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
```

```
            <input type="hidden" asp-for="AppointmentId" />
```

```
            <div class="form-group">
```

```
                <label asp-for="PatientId" class="control-label">Patient Name:</label>
```

```
                <select asp-for="PatientId" class="form-control" asp-
```

```
items="ViewBag.PatientName"></select>
```

```
                <span asp-validation-for="PatientId" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="DoctorId" class="control-label">Doctor Name:</label>
```

```
                <select asp-for="DoctorId" class="form-control" asp-
```

```
items="ViewBag.DoctorName"></select>
```

```
                <span asp-validation-for="DoctorId" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="DateofAppoint" class="control-label">Date of
```

```
Appointment: (YYYY-MM-DD Format)</label>
```

```
                <input asp-for="DateofAppoint" class="form-control" />
```

```
                <span asp-validation-for="DateofAppoint" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="TimeofAppoint" class="control-label">Time of
```

```
Appointment: (24 Hour Format)</label>
```

```
                <input asp-for="TimeofAppoint" class="form-control" />
```

```
                <span asp-validation-for="TimeofAppoint" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label asp-for="StatusofAppoint" class="control-label">Status of
```

```
Appointment:</label>
```

```
                <select asp-for="StatusofAppoint" class="form-control">
```

```
                    <option value="">-- Select Status --</option>
```

```
                    <option value="Pending">Pending</option>
```

```
                    <option value="Ongoing">Ongoing</option>
```

```
                    <option value="Cancelled">Cancelled</option>
```

```
                    <option value="Completed">Completed</option>
```

```
                </select>
```

```
                <span asp-validation-for="StatusofAppoint" class="text-danger"></span>
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <input type="submit" value="Save" class="btn btn-primary" />
```

```
            </div>
```

```
        </form>
```

```
    </div>
```

```
</div>
```

```
<div>
```

```
  <a asp-action="Index">Back to List</a>
```

```
</div>
```

```
@section Scripts {
```

```
  @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
```

```
}
```

# Appointment/Delete

```
@model AppointmentManager.Models.Appointment

@{
    ViewData["Title"] = "Delete";
}

<h1>Delete</h1>

<h3>Are you sure you want to delete this?</h3>
<div>
    <h4>Appointment</h4>
    <hr />
    <dl class="row">
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.DateofAppoint)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.DateofAppoint)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.TimeofAppoint)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.TimeofAppoint)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.StatusofAppoint)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.StatusofAppoint)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Doctor)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Doctor.DoctorId)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Patient)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Patient.PatientId)
        </dd>
    </dl>

    <form asp-action="Delete">
        <input type="hidden" asp-for="AppointmentId" />
        <input type="submit" value="Delete" class="btn btn-danger" /> |
        <a asp-action="Index">Back to List</a>
    </form>
</div>
```

# Appointment/Details

```
@model AppointmentManager.Models.Appointment

@{
    ViewData["Title"] = "Details";
}

<h1>Details</h1>

<div>
    <h4>Appointment</h4>
    <hr />
    <dl class="row">
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.DateofAppoint)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.DateofAppoint)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.TimeofAppoint)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.TimeofAppoint)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.StatusofAppoint)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.StatusofAppoint)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Doctor)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Doctor.DoctorId)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Patient)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Patient.PatientId)
        </dd>
    </dl>
</div>
<div>
    <a asp-action="Edit" asp-route-id="@Model?.AppointmentId">Edit</a> |
    <a asp-action="Index">Back to List</a>
</div>
```

# Program.cs

```
using AppointmentManager.Data;
using Microsoft.EntityFrameworkCore;
using System;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<AppDbContext>(options =>
options.UseMySQL(builder.Configuration.GetConnectionString("DefaultConnection"),
ServerVersion.AutoDetect((builder.Configuration.GetConnectionString("DefaultConnection"
)))));

// Add services to the container.
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production
    scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

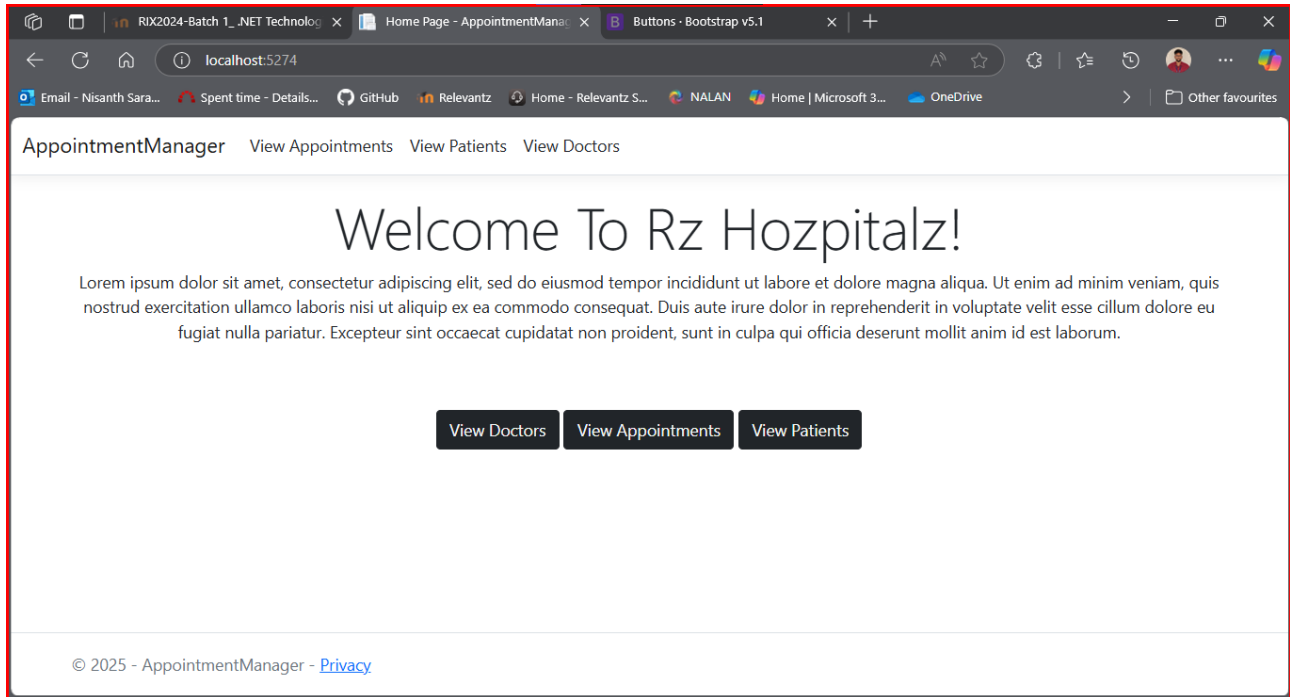
app.Run();
```

## appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }, //Connection Strings
  "ConnectionStrings": {
    "DefaultConnection":
"server=localhost;user=root;password=root;database=HospitalDb;port=3306;"
  },
  "AllowedHosts": "*"
}
```

# Output Screens

## Home/Index



# Patient

## Create

**Create Patient**

Patient Name:

Email:

Contact Number:

Date of Birth: (YYYY-MM-DD Format)

Gender:

-- Select Gender --

Male

Female

Other

## Index

AppointmentManager View Appointments View Patients View Doctors

**Index**

Create New Record

| PatientName | PatientEmail                    | PatientPhone | PatientDob | PatientGender |                                                                         |
|-------------|---------------------------------|--------------|------------|---------------|-------------------------------------------------------------------------|
| Nisanth     | nisanth.saravanan@relevantz.com | 1234567890   | 30/06/2004 | Male          | <a href="#">Edit</a>   <a href="#">Details</a>   <a href="#">Delete</a> |
| Rishi       | Sample@gmail.com                | 1234567890   | 30/01/2003 | Male          | <a href="#">Edit</a>   <a href="#">Details</a>   <a href="#">Delete</a> |
| Maddy       | Sample@relevantz.com            | 1234567890   | 30/06/2004 | Male          | <a href="#">Edit</a>   <a href="#">Details</a>   <a href="#">Delete</a> |
| Bangroos    | Sample@relevantz.com            | 1234567890   | 30/06/2004 | Male          | <a href="#">Edit</a>   <a href="#">Details</a>   <a href="#">Delete</a> |

© 2025 - AppointmentManager - [Privacy](#)



## Edit

## Before

|          |                      |            |            |      |      |         |        |
|----------|----------------------|------------|------------|------|------|---------|--------|
| Bangroos | Sample@relevantz.com | 1234567890 | 30/06/2004 | Male | Edit | Details | Delete |
|----------|----------------------|------------|------------|------|------|---------|--------|

The screenshot shows a web browser window with the URL `localhost:5274/Patient/Edit/9`. The browser tabs include 'RUX2024-Batch 1\_ .NET Technolog...', 'Edit - AppointmentManager', and 'Buttons - Bootstrap v5.1'. The browser's address bar shows the URL. The browser's toolbar includes navigation buttons, a search bar, and a list of bookmarks. The browser's content area shows the 'AppointmentManager' application. The application has a navigation bar with links: 'AppointmentManager', 'View Appointments', 'View Patients', and 'View Doctors'. The main content area has a heading 'Edit Patient'. Below the heading is a form with the following fields: 'Patient Name:' with a text input containing 'Nithish Bangaru'; 'Email:' with a text input containing 'Sample@relevantz.com'; 'Contact Number:' with a text input containing '1234567890'; 'Date of Birth: (YYYY-MM-DD Format)' with a date picker showing '27-07-2004'; and 'Gender:' with a text input containing 'Male'. Below the form is a blue 'Save' button and a blue link 'Back to List'.

## After

|                 |                      |            |            |      |      |         |        |
|-----------------|----------------------|------------|------------|------|------|---------|--------|
| Nithish Bangaru | Sample@relevantz.com | 1234567890 | 27/07/2004 | Male | Edit | Details | Delete |
|-----------------|----------------------|------------|------------|------|------|---------|--------|

# Details

RIX2024-Batch 1\_ .NET Technolog

Details - AppointmentManager

Buttons - Bootstrap v5.1

localhost:5274/Patient/Details/6

🔊

☆

⚙️

🔖

🔄

👤

⋮

🌈

Email - Nisanth Sara...Spent time - Details...GitHubRelevantzHome - Relevantz S...NALANHome | Microsoft 3...OneDrive>Other favourites

AppointmentManagerView AppointmentsView PatientsView Doctors

## Details

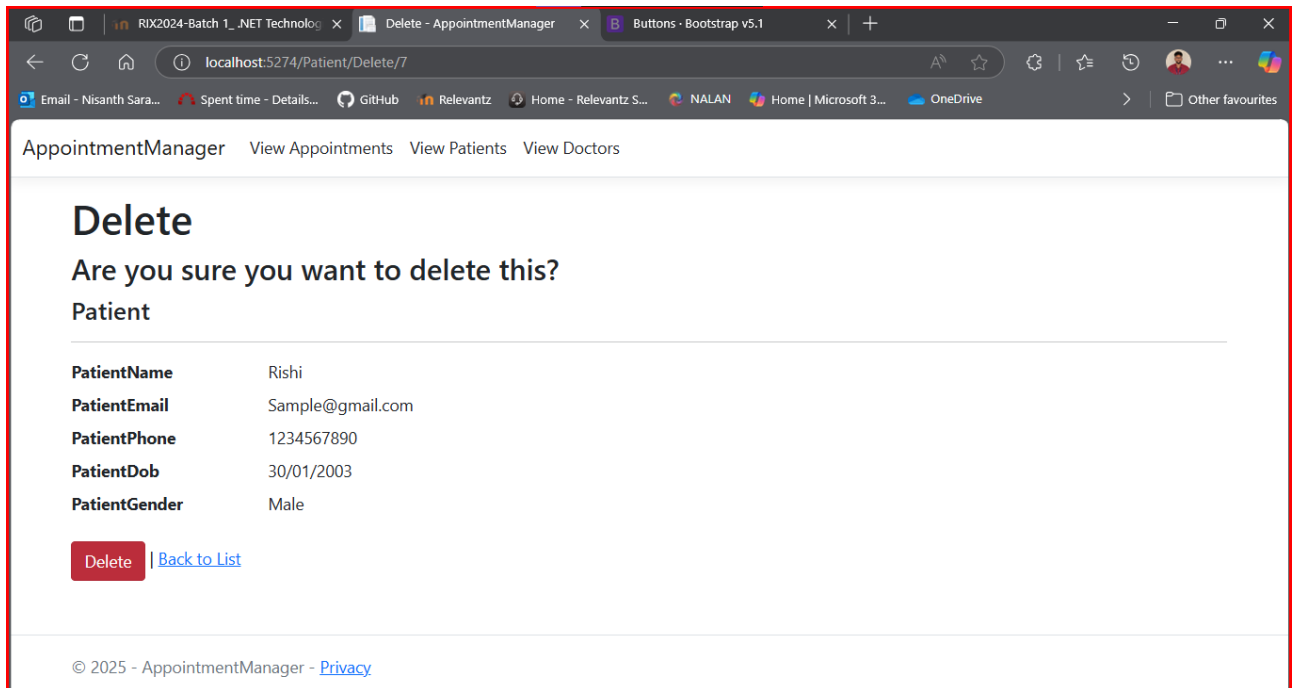
### Patient

|               |                                 |
|---------------|---------------------------------|
| PatientName   | Nisanth                         |
| PatientEmail  | nisanth.saravanan@relevantz.com |
| PatientPhone  | 1234567890                      |
| PatientDob    | 30/06/2004                      |
| PatientGender | Male                            |

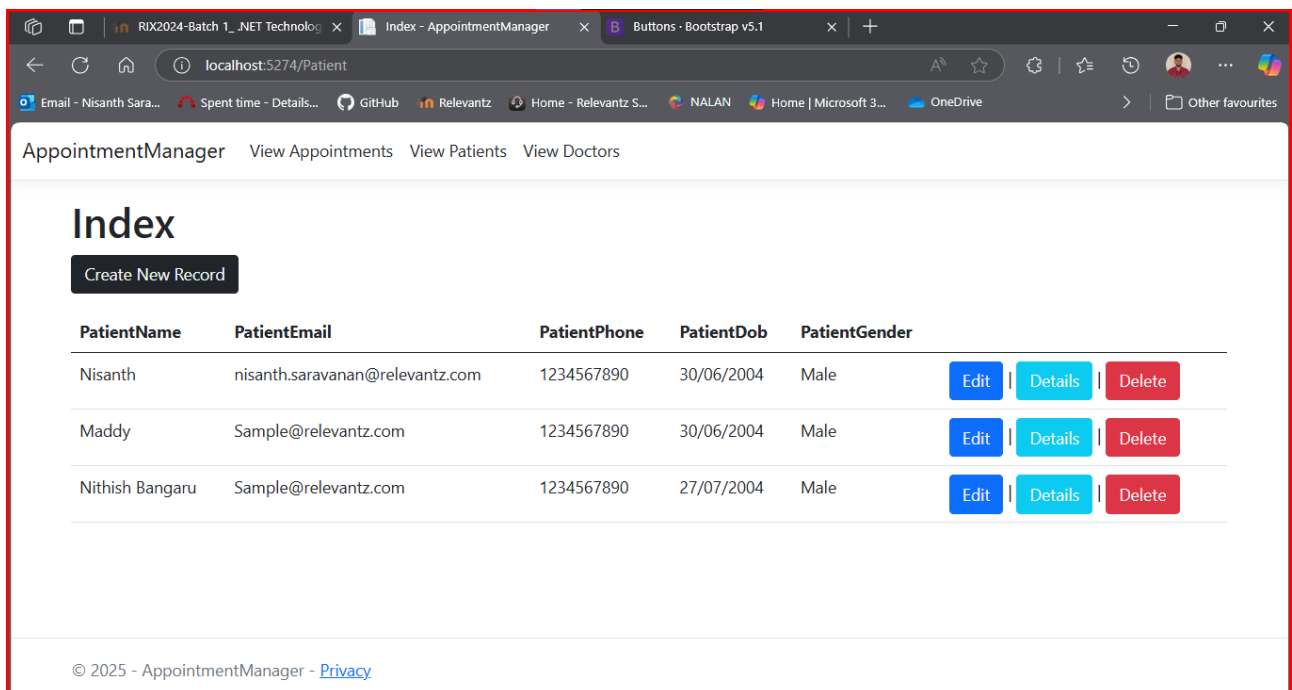
[Edit](#) | [Back to List](#)

© 2025 - AppointmentManager - [Privacy](#)

# Delete



# Post Deletion



# Doctor

## Create

AppointmentManager

View Appointments

View Patients

View Doctors

Create

Doctor

Doctor Name:

Rishi

Specialization:

Eyes and ears

Availability:

-- Select Availability --

-- Select Availability --

Available

Unavailable

© 2025 - AppointmentManager - [Privacy](#)

## Index

AppointmentManager

View Appointments

View Patients

View Doctors

Index

Create New Record

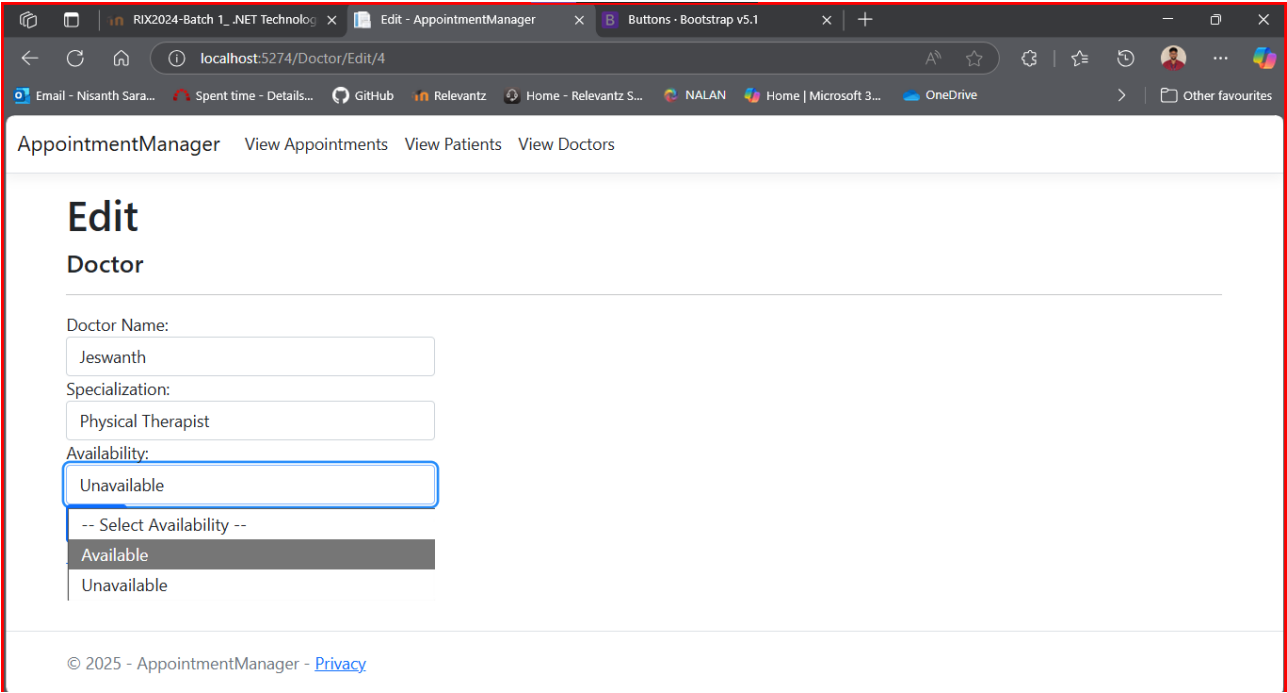
| DoctorName | Specialization     | Availability |                              |
|------------|--------------------|--------------|------------------------------|
| Rishi      | Eyes and ears      | Available    | <div>EditDetailsDelete</div> |
| Jeswanth   | Physical Therapist | Unavailable  | <div>EditDetailsDelete</div> |
| Oldie      | Ageing             | Unavailable  | <div>EditDetailsDelete</div> |

© 2025 - AppointmentManager - [Privacy](#)

# Edit

## Before

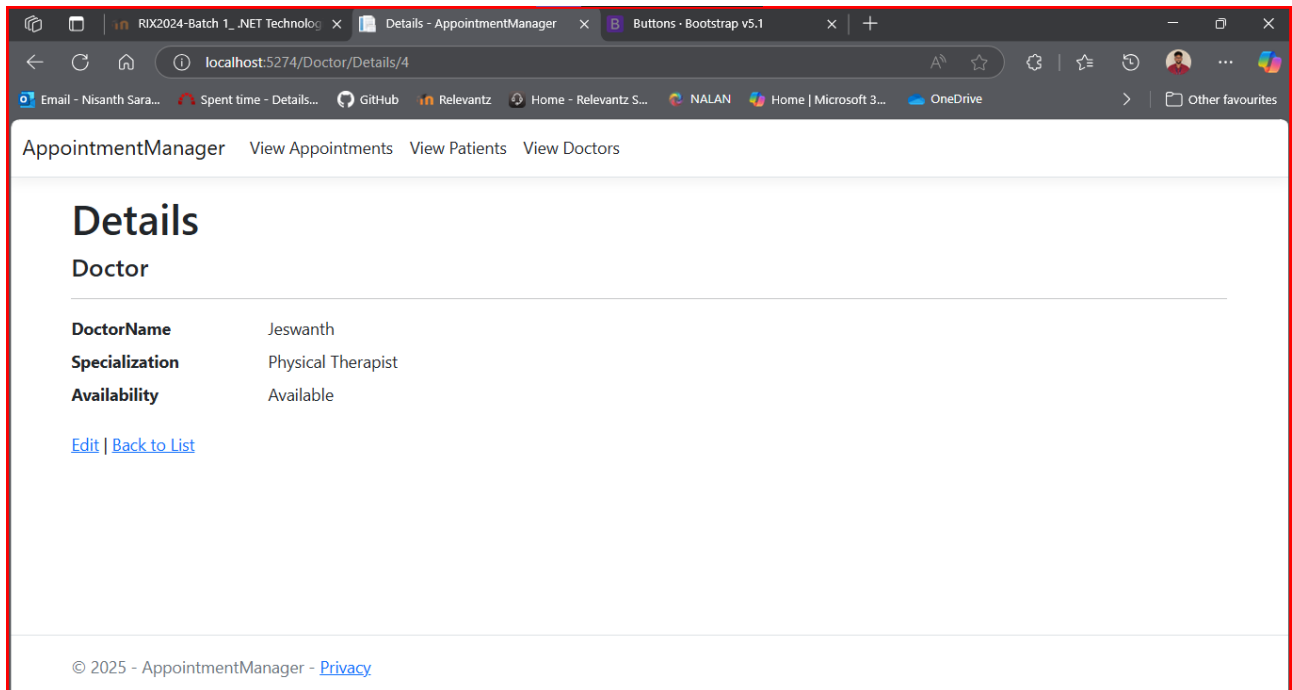
|          |                    |             |                      |                         |                        |
|----------|--------------------|-------------|----------------------|-------------------------|------------------------|
| Jeswanth | Physical Therapist | Unavailable | <a href="#">Edit</a> | <a href="#">Details</a> | <a href="#">Delete</a> |
|----------|--------------------|-------------|----------------------|-------------------------|------------------------|



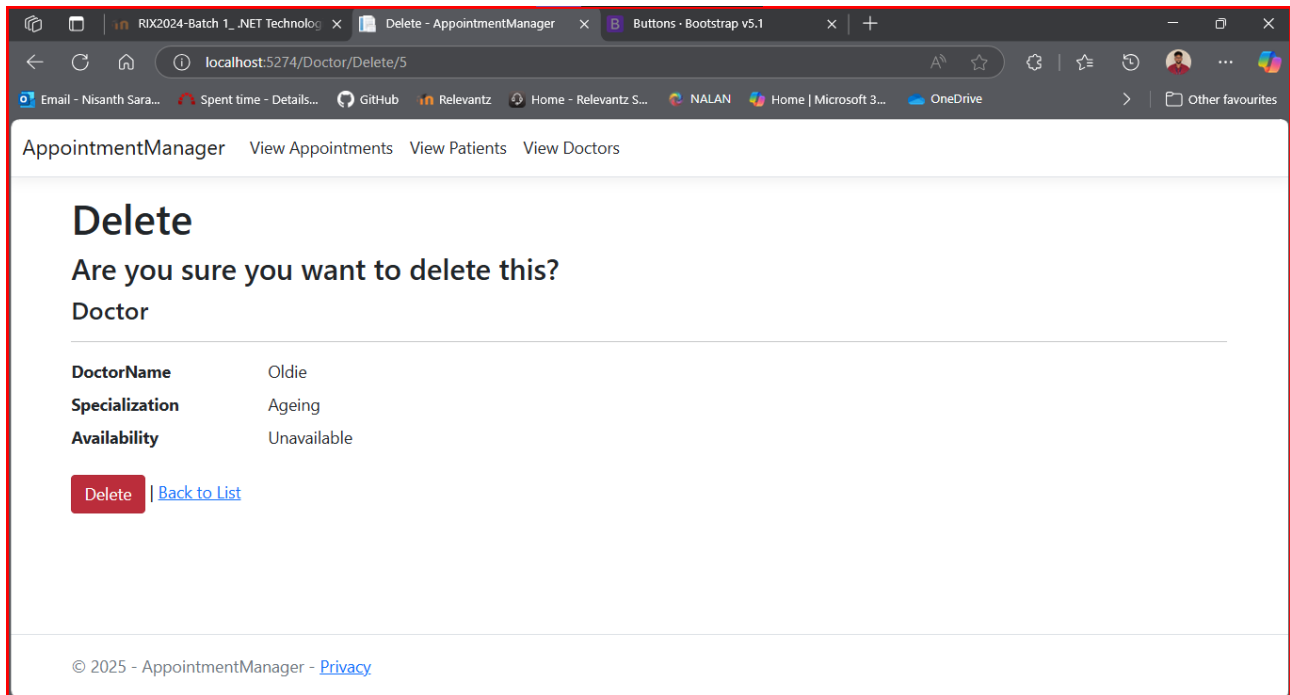
## After

|          |                    |           |                      |                         |                        |
|----------|--------------------|-----------|----------------------|-------------------------|------------------------|
| Jeswanth | Physical Therapist | Available | <a href="#">Edit</a> | <a href="#">Details</a> | <a href="#">Delete</a> |
|----------|--------------------|-----------|----------------------|-------------------------|------------------------|

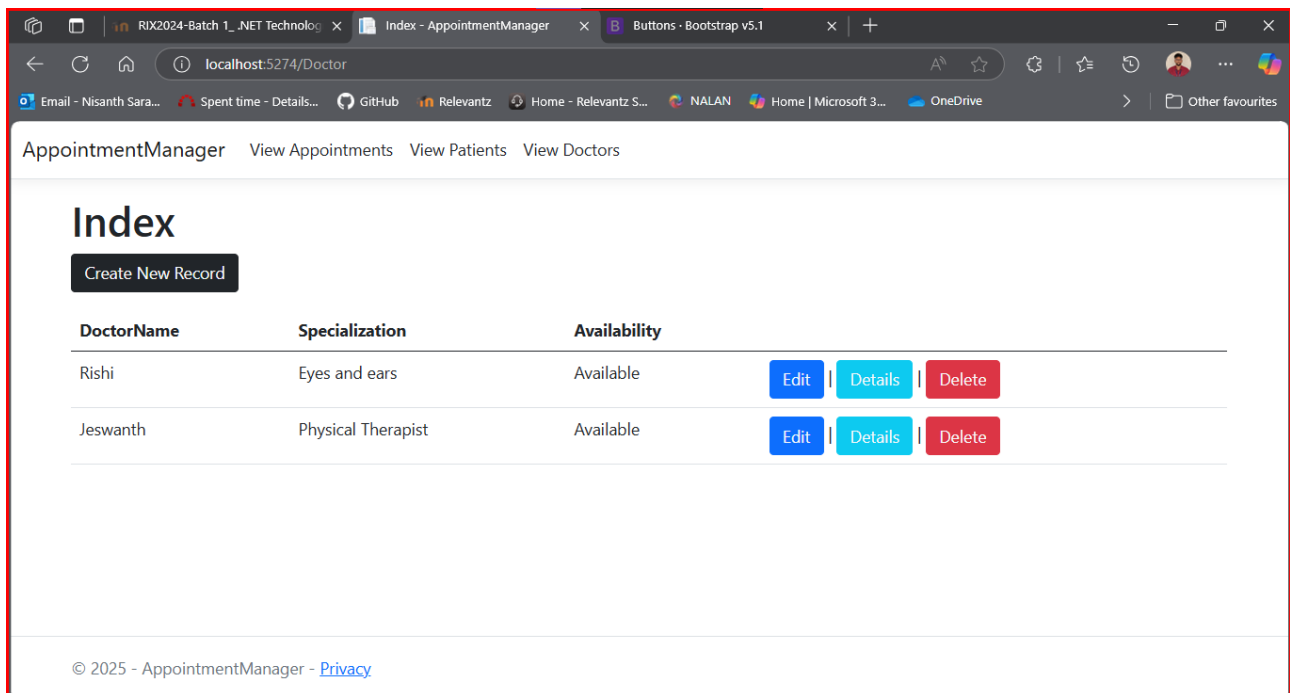
# Details



# Delete



# After Deletion



## Appointment

Create

# Create

## Appointment

---

Patient Name:

Nisanth

Nisanth

Maddy

Nithish Bangaru

# Create

## Appointment

---

Patient Name:

Maddy

Doctor Name:

Rishi

Rishi

Jeswanth

Time of Appointment: (24 Hour Format)



RIX2024-Batch 1\_ .NET Technolog

Create - AppointmentManager

Buttons - Bootstrap v5.1

localhost:5274/Appointment/Create

Email - Nisanth Sara...Spent time - Details...GitHubRelevantzHome - Relevantz S...NALANHome | Microsoft 3...OneDriveOther favourites

# Create

## Appointment

Patient Name:

Maddy

Doctor Name:

Rishi

-- Select Status --

Pending

Ongoing

Cancelled

Completed

-- Select Status --

Create

[Back to List](#)

© 2025 - AppointmentManager - [Privacy](#)

RIX2024-Batch 1\_ .NET Technolog

Create - AppointmentManager

Buttons - Bootstrap v5.1

localhost:5274/Appointment/Create

Email - Nisanth Sara...Spent time - Details...GitHubRelevantzHome - Relevantz S...NALANHome | Microsoft 3...OneDriveOther favourites

# Create

## Appointment

Patient Name:

Maddy

Doctor Name:

Rishi

Date of Appointment: (YYYY-MM-DD Format)

2025-08-30

Time of Appointment: (24 Hour Format)

15:04

Status of Appointment:

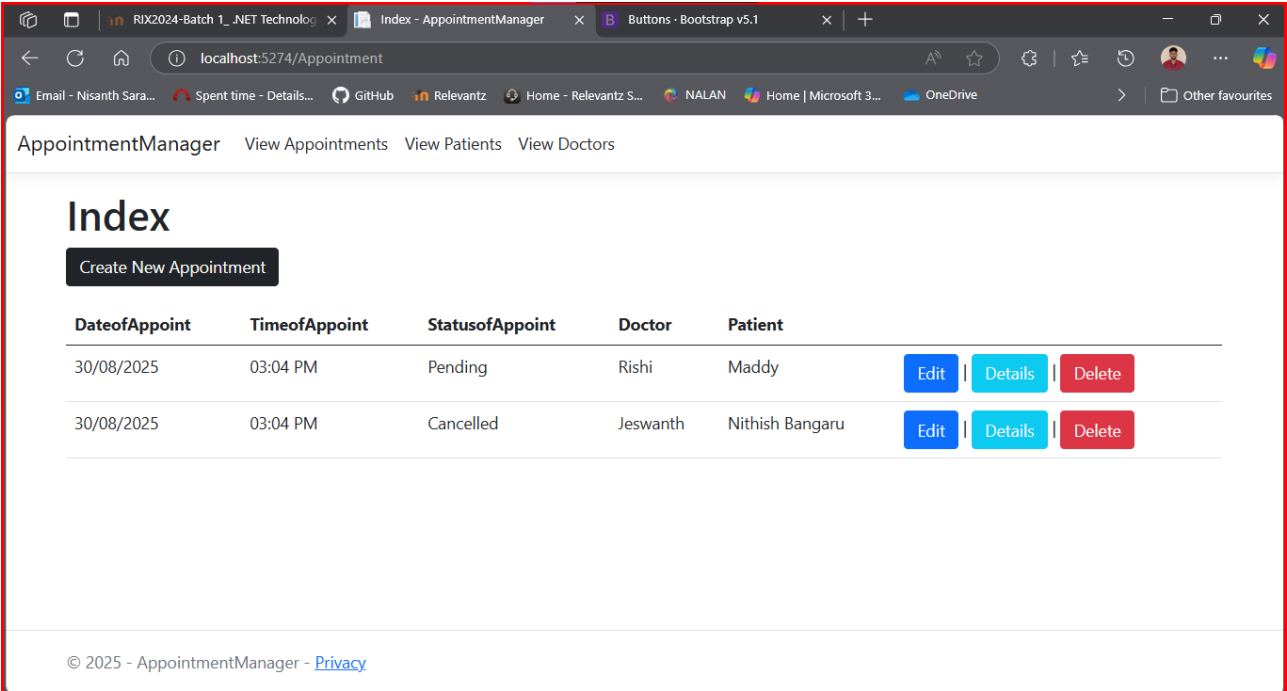
Pending

Create

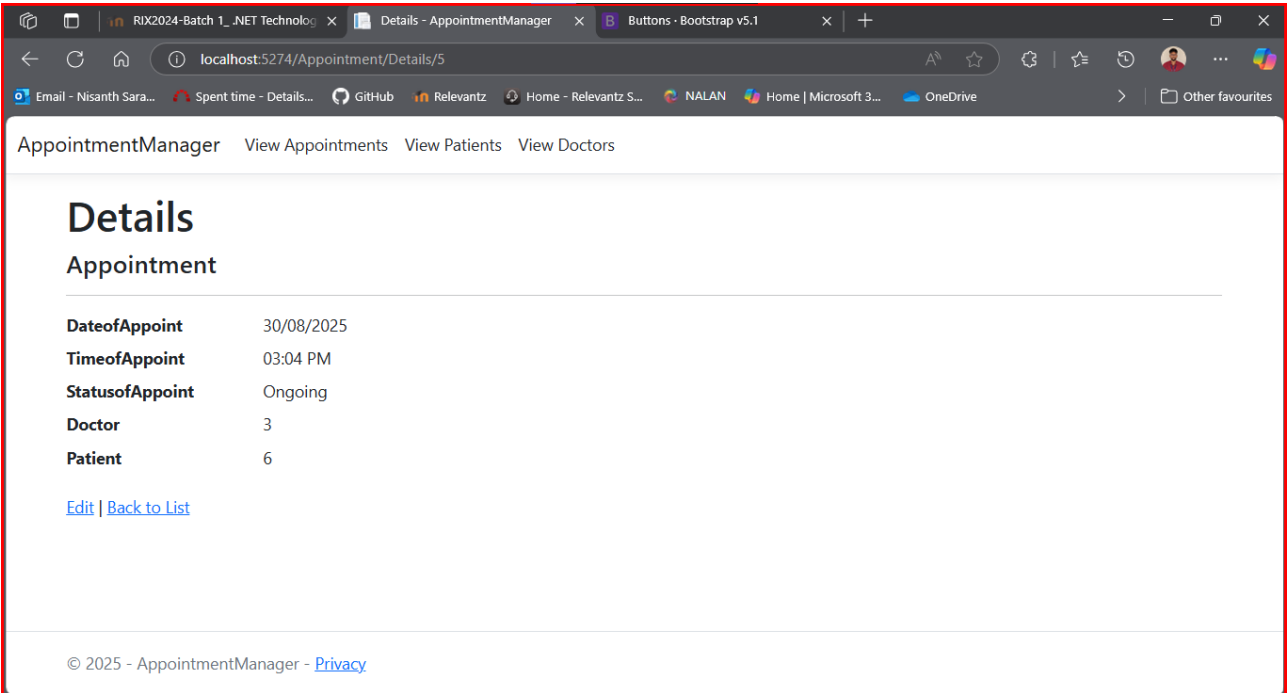
[Back to List](#)

© 2025 - AppointmentManager - [Privacy](#)

# Index



# Details



## Edit

## Before

### Index

Create New Appointment

| DateofAppoint | TimeofAppoint | StatusofAppoint | DoctorName | PatientName |                                                                         |
|---------------|---------------|-----------------|------------|-------------|-------------------------------------------------------------------------|
| 30/08/2025    | 03:04 PM      | Ongoing         | Rishi      | Nisanth     | <a href="#">Edit</a>   <a href="#">Details</a>   <a href="#">Delete</a> |
| 30/08/2025    | 03:04 PM      | Completed       | Jeswanth   | Maddy       | <a href="#">Edit</a>   <a href="#">Details</a>   <a href="#">Delete</a> |

AppointmentManager View Appointments View Patients View Doctors

## Edit Appointment

Patient Name:

Doctor Name:

-- Select Status --  
Pending  
Ongoing  
Cancelled  
Completed

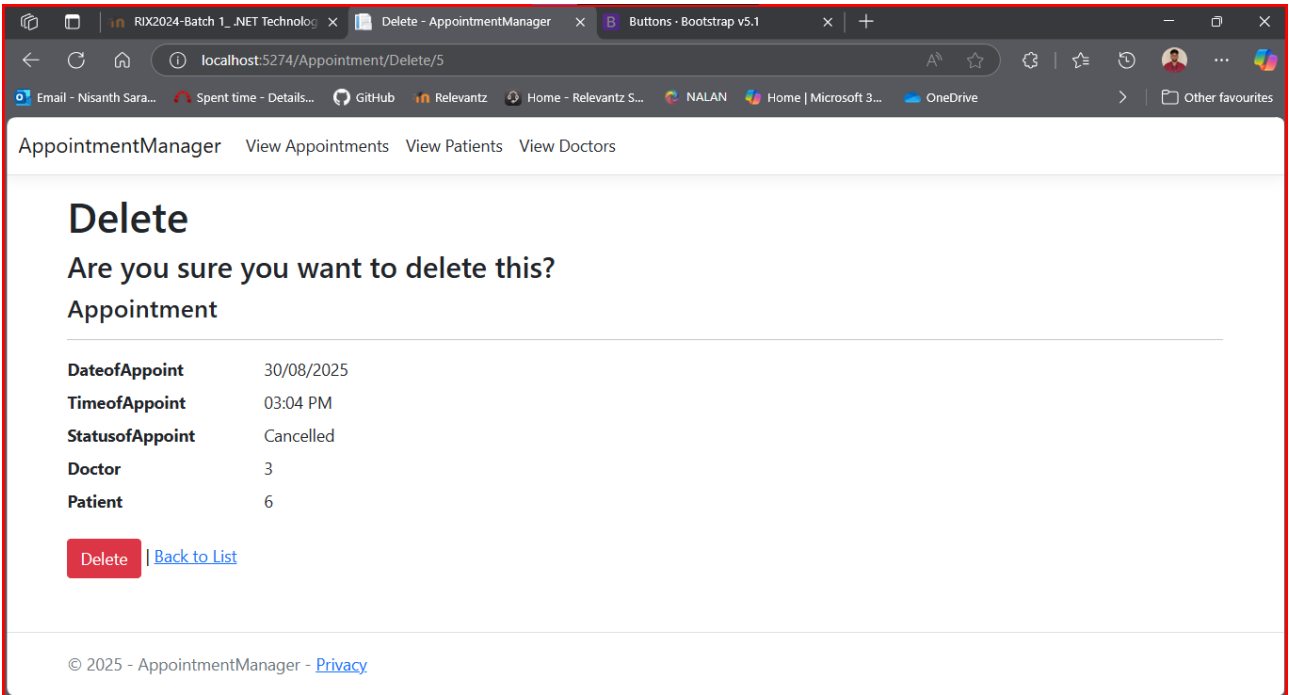
[Save](#)

[Back to List](#)

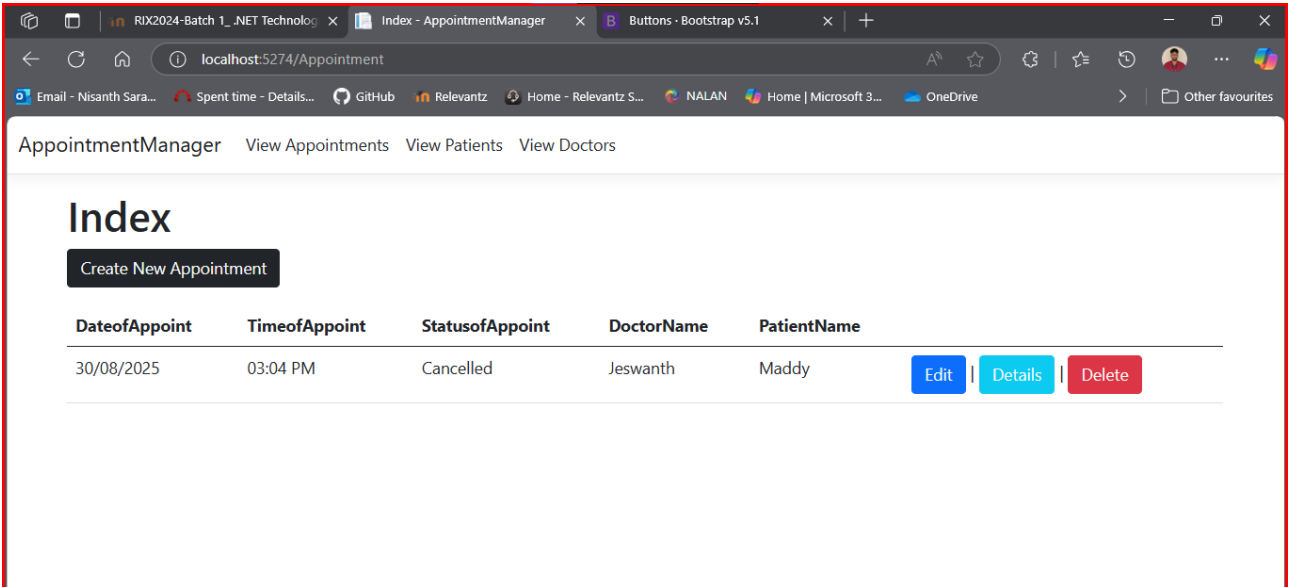
## After

| DateofAppoint | TimeofAppoint | StatusofAppoint | DoctorName | PatientName |                                                                         |
|---------------|---------------|-----------------|------------|-------------|-------------------------------------------------------------------------|
| 30/08/2025    | 03:04 PM      | Cancelled       | Rishi      | Nisanth     | <a href="#">Edit</a>   <a href="#">Details</a>   <a href="#">Delete</a> |
| 30/08/2025    | 03:04 PM      | Cancelled       | Jeswanth   | Maddy       | <a href="#">Edit</a>   <a href="#">Details</a>   <a href="#">Delete</a> |

# Delete



# Post Deletion



=====